Tarea 2: Cerradura convexa

CC5502 Geometría Computacional — ANALISIS

Matias Saavedra

Octubre 2026

1. Objetivo y preguntas

Se implementaron dos algoritmos de cerradura convexa en C++, sobre las clases `Punto`, `Vector` y `Poligono` de la Tarea 1: el Incremental (ordenar, construir el *upper hull* y devolverse construyendo el *lower hull*) y el Random Incremental de la sección 2 de las notas de CMU que cita el enunciado [1]. Los dos son $O(n \log n)$ (el segundo en esperanza), así que el análisis asintótico no dice cuál conviene: lo deciden las constantes y cómo depende cada uno de la forma de la entrada. Siguiendo la Lectura 2 [3], las preguntas se fijaron antes de medir:

P1 ¿Crece el tiempo de cada algoritmo como $n \log n$?

P2 Con puntos aleatorios uniformes, ¿cuál es más rápido y por cuánto?

P3 ¿Cómo depende el tiempo del porcentaje de puntos que forman la cerradura? ¿Hay un porcentaje desde el cual conviene uno u otro, y se puede determinar?

P4 ¿Qué cambia entre coordenadas enteras (`int`) y reales (`double`)?

P5 ¿Coinciden las cerraduras cuando hay puntos colineales en el borde, y cuando no los hay?

2. Implementación

2.1. Estructura

El código está en los headers de `include/`, porque todo es template (`T` = `int` o `double`). Siguiendo el patrón Strategy, `EstrategiaCerradura<T>` declara el método

```
virtual Poligono<T> apply(const std::vector<Punto<T>> &nube) = 0;
```

y de ella heredan `IncrementalAlgorithm` y `RandomIncrementalAlgorithm`. Los generadores de nubes siguen el mismo patrón: `GeneradorPuntos<T>` declara `generar(n, rng)` y de él heredan `GeneradorUniforme`, `GeneradorCircunferencia` y `GeneradorCuadrado`. El programa del experimento solo recorre listas de estrategias y generadores.

Los dos algoritmos usan un único predicado geométrico, `orientacion(a, b, c)`: el signo del producto cruz $(b-a) \times (c-a)$, calculado con `Vector::productoCruz` de la Tarea 1. Con `T` entero las coordenadas se promueven a `long long` antes de restar, y el signo es exacto mientras $|x|, |y| < 2^{30}$; con `double` el producto se redondea. De la Tarea 1 se agregaron `Punto::operator<` (orden lexicográfico), un constructor vacío de `Poligono`, `Poligono::normalizar` y `Poligono::operator==`.

2.2. Incremental

Ordena los puntos por x (y por y a igual x), elimina los repetidos y recorre la lista de izquierda a derecha construyendo el *upper hull*: como el punto nuevo es el extremo derecho de los ya vistos, siempre es vértice de su cerradura, y basta agregarlo al final y sacar los últimos mientras los tres últimos no formen una vuelta a la derecha. Luego se devuelve de derecha a izquierda con la misma regla para el *lower hull* ([2], sección 1.1). Cada punto entra y sale de la cadena a lo más una vez por recorrido, así que tras el orden $O(n \log n)$ el resto es $O(n)$, y el costo *no depende* de cuántos puntos quedan en la cerradura.

2.3. Random Incremental

Baraja los puntos (`std::shuffle` con `std::mt19937_64` y una semilla fija), toma un triángulo no degenerado y agrega los demás puntos en ese orden. La dificultad es saber, para cada punto nuevo, si está dentro y qué aristas ve. Como en [1], se fija un punto C interior (el centroide del triángulo inicial; la cerradura solo crece, así que C sigue dentro) y cada punto que falta por insertar guarda la arista que cruza el rayo $C \to q$. Esa arista es visible desde q exactamente cuando q está fuera, de modo que q *está fuera si y solo si está estrictamente a la derecha de su arista*: una sola orientación. Cada arista guarda, a su vez, la "cubeta" de puntos que la apuntan. Al insertar q:

1. se recorre la cerradura desde la arista de q hacia ambos lados mientras las aristas sean visibles, hasta los vértices l y r;
2. se sacan las cubetas de las aristas visibles y la cadena $l \dots r$ se reemplaza por $l \to q \to r$;
3. cada punto de esas cubetas cruza ahora $l \to q$ o $q \to r$, según de qué lado del rayo $C \to q$ esté, y pasa a esa cubeta; si ya no ve su nueva arista, está dentro de la cerradura y se descarta para siempre.

Por análisis hacia atrás, el número esperado de reasignaciones es a lo más $2nH_n \approx 2n \ln n$ para cualquier entrada [1]: la esperanza es sobre el barajado, no sobre los datos. El descarte del paso 3 no está en las notas; no cambia la cota pero hace que el trabajo dependa de cuántos puntos siguen fuera de la cerradura, que es lo que mide la pregunta P3. El método `getReasignaciones()` entrega el contador para el experimento.

Calidad de la implementación. Moret advierte que una implementación descuidada cambia las conclusiones [3]. Se midieron tres versiones del Random Incremental con la misma nube de $n = 10^6$ puntos (`resultados/versiones_random_incremental.txt`). Guardar las cubetas como listas enlazadas a través de un arreglo por punto tardaba 1 997 ms con todos los puntos en la cerradura; agrupar los campos de cada punto en un `struct` casi no cambió el tiempo (1 923 ms), y guardar cada cubeta en un `std::vector<int>` contiguo lo bajó a 1 624 ms (con $p = 10$ %: 371, 445 y 334 ms). Se usó esta última, que además es la más simple. El perfil (`valgrind -tool=callgrind`) muestra que el costo restante son las operaciones mismas, unas 125 instrucciones por reasignación (dos orientaciones, la cubeta nueva y su crecimiento), más la latencia de memoria cuando n ya no cabe en el caché.

2.4. Supuestos y decisiones

- **Salida canónica.** Ambos algoritmos entregan solo los vértices extremos (un punto colineal sobre una arista no es vértice, y los repetidos aparecen una vez), en sentido antihorario y desde el menor vértice en orden lexicográfico. Así "cerraduras idénticas" es comparar vértice a vértice (`Poligono::operator==`). Con todos los puntos colineales la cerradura es el segmento entre los extremos.

- **Colineales en el Incremental.** Se saca el último punto de la cadena cuando la orientación es ≥ 0, no solo > 0.
- **Colineales en el Random Incremental.** La prueba "q está fuera" es estricta (un punto sobre el borde está dentro), pero el recorrido del paso 1 no: también elimina una arista cuya recta pasa por q. Si q está en la prolongación de una arista, el vértice que comparten deja de ser extremo; con un recorrido estricto quedaría como vértice colineal y las cerraduras no coincidirían. Las notas suponen que no hay tres puntos colineales; esta regla es lo que hace falta sin ese supuesto.
- **Triángulo inicial.** Se toma el primer punto distinto del primero y luego el primero no colineal con ambos: con coordenadas enteras o colineales los tres primeros puntos del barajado pueden no servir.
- C **exacto.** El centroide $C = (a + b + c)/3$ no tiene coordenadas enteras, y un triángulo entero puede no tener ningún punto entero en su interior. Las orientaciones respecto de C se calculan con $3p - (a + b + c)$, que es 3 veces el vector $C \to p$. Con `T` entero esto es exacto en `long long` si $|x|, |y| < 2^{28}$; los generadores usan $|x|, |y| \leq 1.42 \cdot 10^8 < 2^{28}$.
- **Aritmética inexacta.** Con `double` no se garantiza que ambos algoritmos coincidan. El recorrido del paso 1 tiene topes para no dar la vuelta completa a la cerradura si los signos se contradicen.

2.5. Generadores

Los tres trabajan en `double`, con coordenadas de hasta $R = 10^8$, y con `T` entero redondean al entero más cercano: con la misma semilla, la nube `int` es la nube `double` redondeada, y la única diferencia entre los dos tipos es el redondeo.

- `GeneradorUniforme`: n puntos uniformes en $[-R, R]^2$. Su cerradura tiene en esperanza unos $\frac{8}{3} \ln n$ vértices [4], unos 37 para $n = 10^6$ (se midió una mediana de 36): es el caso de un porcentaje muy pequeño en la cerradura.
- `GeneradorCircunferencia(p)`: $k = \max(3, pn)$ puntos equiespaciados sobre la circunferencia de radio R, con un desfase aleatorio, y el resto uniforme en el disco inscrito en ese k-ágono, de radio $0.999\, R \cos(\pi/k)$. Los interiores quedan estrictamente dentro del polígono, y no solo del círculo: si se sortearan en el disco de radio R, los que caen entre una arista y el arco serían vértices y el porcentaje real sería mayor que el pedido. Así, con `double` la cerradura tiene exactamente k vértices, sin colineales.
- `GeneradorCuadrado(p, θ)`: las 4 esquinas y $\max(0, pn - 4)$ puntos más sobre los lados del cuadrado $[-R, R]^2$ rotado en θ, y el resto dentro. Con $\theta = 0$ los lados son horizontales y verticales y la colinealidad es exacta en ambos tipos; con $\theta = 30°$ los puntos están sobre la recta solo en los reales: en `double` quedan a un redondeo de ella, y en `int` a medio entero.

3. Diseño experimental

En total son 1 140 casos (Tabla 1); en cada uno se genera una nube y se miden ambos algoritmos sobre esa misma nube.

- **Medidas.** El tiempo de `apply` (que incluye la copia de la entrada y el orden o el barajado; no incluye generar la nube), el tamaño h de cada cerradura, si las dos son idénticas y, para el Random Incremental, el número de reasignaciones. Esta última es una medida estructural [3]: se reproduce exactamente y explica el tiempo.

Tabla 1: Partes del experimento. Cada caso se mide con `int` y con `double`.

Parte	Generador	n	Repeticiones
Escalamiento	uniforme; circunferencia 100 %	$1, 2, 5 \cdot 10^4$; $1, 2, 5 \cdot 10^5$; 10^6	10 nubes
Porcentaje	circunferencia con $p = 0.1$, 0.2, 0.5, 1, 2, 5, 10, 20, 50 y 100 %	10^4, 10^5, 10^6	10 nubes
Colinealidad	cuadrado 0° y 30°; circunferencia ($p = 10$ %)	10^4, 10^5, 10^6	10 nubes
Varianza	uniforme; circunferencia 100 %	10^6	1 nube, 20 semillas

- **Tiempo.** Con `std::chrono::steady_clock`. Cada medición repite `apply` hasta juntar al menos 50 ms y divide, para que la resolución del reloj no domine con n pequeño. Se reporta la mediana de las 10 nubes y, en los gráficos, la banda entre los cuartiles.
- **Confusores controlados.** Ambos algoritmos sobre la misma nube; el orden de medición se alterna entre nubes; el Random Incremental libera su memoria de trabajo al terminar, como el Incremental, para que las repeticiones reserven memoria igual que la primera; el proceso se fija a un núcleo (`taskset -c 3`) en una máquina sin otra carga.
- **Variabilidad.** La parte "varianza" mide una misma nube con 20 semillas del barajado: separa la variabilidad propia del Random Incremental de la que viene de cambiar de nube.
- **Reproducibilidad.** Las semillas son fijas (`std::mt19937_64`; la nube i de tamaño n usa la semilla $1\,000\,003\,i + n$ y el algoritmo la semilla $i + 1$). h, las reasignaciones y las coincidencias se reproducen exactamente.
- **Máquina.** Intel Core i7-9750H (6 núcleos, 2.6 GHz base y hasta 4.5 GHz, caché L3 de 12 MiB), 16 GB de RAM, Linux 7.2.9 (Arch), g++ 16.2.1 con `-O3 -DNDEBUG` (`Release` de CMake), C++17. El regulador de frecuencia (`powersave`, con turbo) no se fijó: es la principal fuente de ruido en los tiempos.

4. Resultados

4.1. Las cerraduras coinciden

En 1 110 de los 1 140 casos las dos cerraduras son idénticas: 280/280 en el escalamiento, 600/600 en el porcentaje, 80/80 en la varianza y 150/180 en la colinealidad. Las 30 diferencias son todas del mismo caso, el cuadrado rotado en 30° con `double`, y se analizan en la Sección 4.6.

4.2. Escalamiento con n (P1)

La Figura 1 confirma el $n \log n$ del Incremental: entre $n = 10^4$ y 10^6 tarda entre 7.3 y 7.7 ns por $n \log_2 n$ con puntos uniformes, y lo mismo con todos los puntos en la cerradura: su costo no depende de h.

El Random Incremental con todos los puntos en la cerradura es el peor caso de su análisis. La cantidad de operaciones sí crece como $n \log n$: con $n = 10^6$ hace 22 840 921 reasignaciones ($22.8\,n$), bajo la cota $2nH_n = 28\,785\,453$, y en la Figura 3 las reasignaciones por punto siguen a $2 \ln n$. Pero el tiempo por $n \log_2 n$ sube de 32 a 74 ns: lo que crece es el costo de *cada* reasignación, de 24–36 ns con $n \le 10^5$ a hasta 70 ns con $n = 10^6$. El quiebre está entre $2 \cdot 10^5$ y $5 \cdot 10^5$ puntos, donde los datos del algoritmo (unos 60 bytes por punto entre el punto, sus enlaces y su cubeta) dejan de caber en los 12 MiB del caché L3, y cada reasignación toca un punto en una posición aleatoria de la memoria. El Incremental no sufre ese efecto: `std::sort` y los recorridos acceden a la memoria en orden.

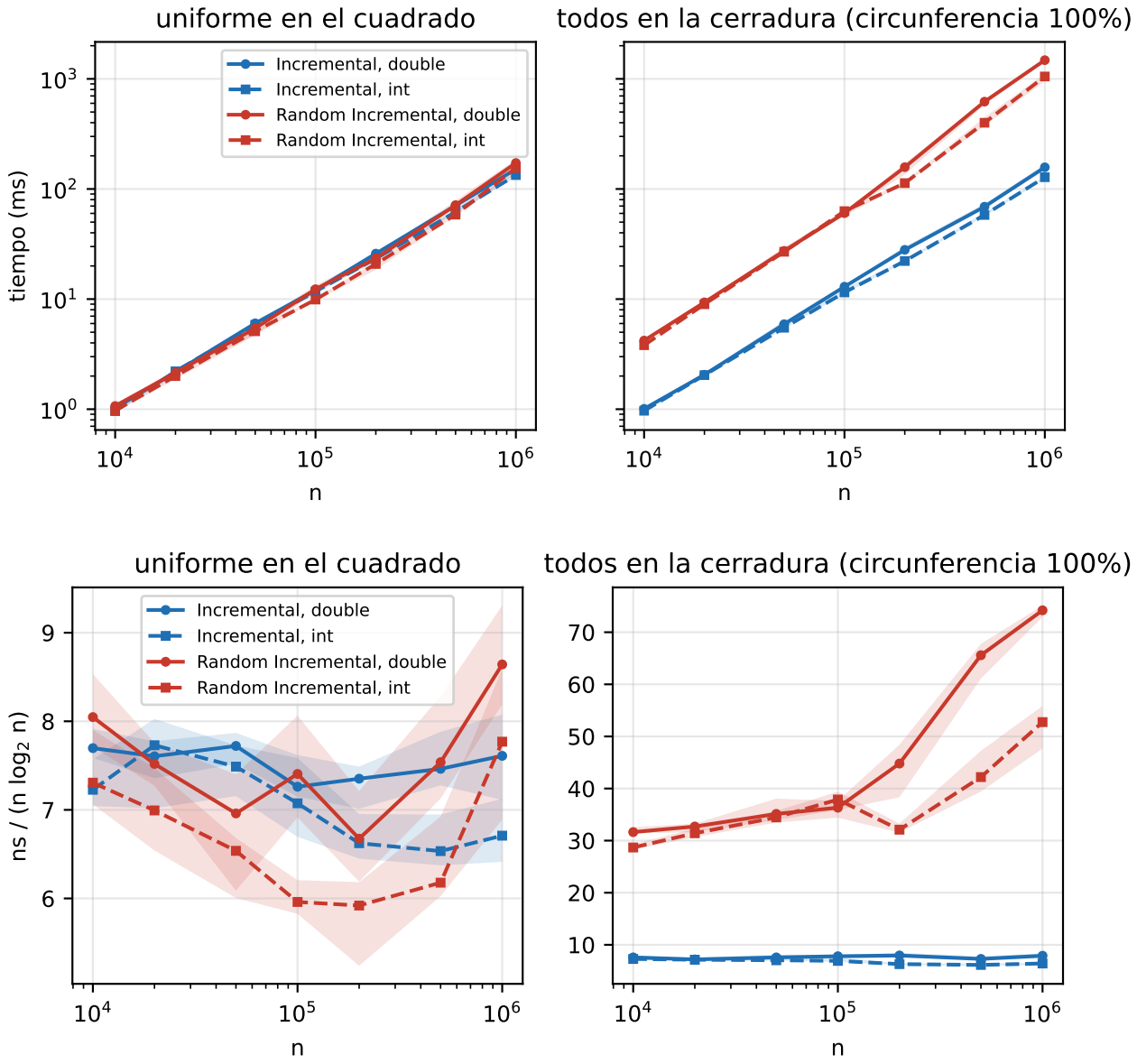


Figura 1: Arriba: tiempo de `apply` contra n (mediana de 10 nubes y banda intercuartil). Abajo: el mismo tiempo dividido por $n \log_2 n$; una curva horizontal es un crecimiento $n \log n$.

Con `int` y todos los puntos "en la cerradura", la curva del Random Incremental se dobla hacia abajo desde $n = 2 \cdot 10^5$: no es el algoritmo sino la entrada, que pierde vértices al redondear (Sección 4.5).

4.3. Porcentaje de puntos en la cerradura (P2 y P3)

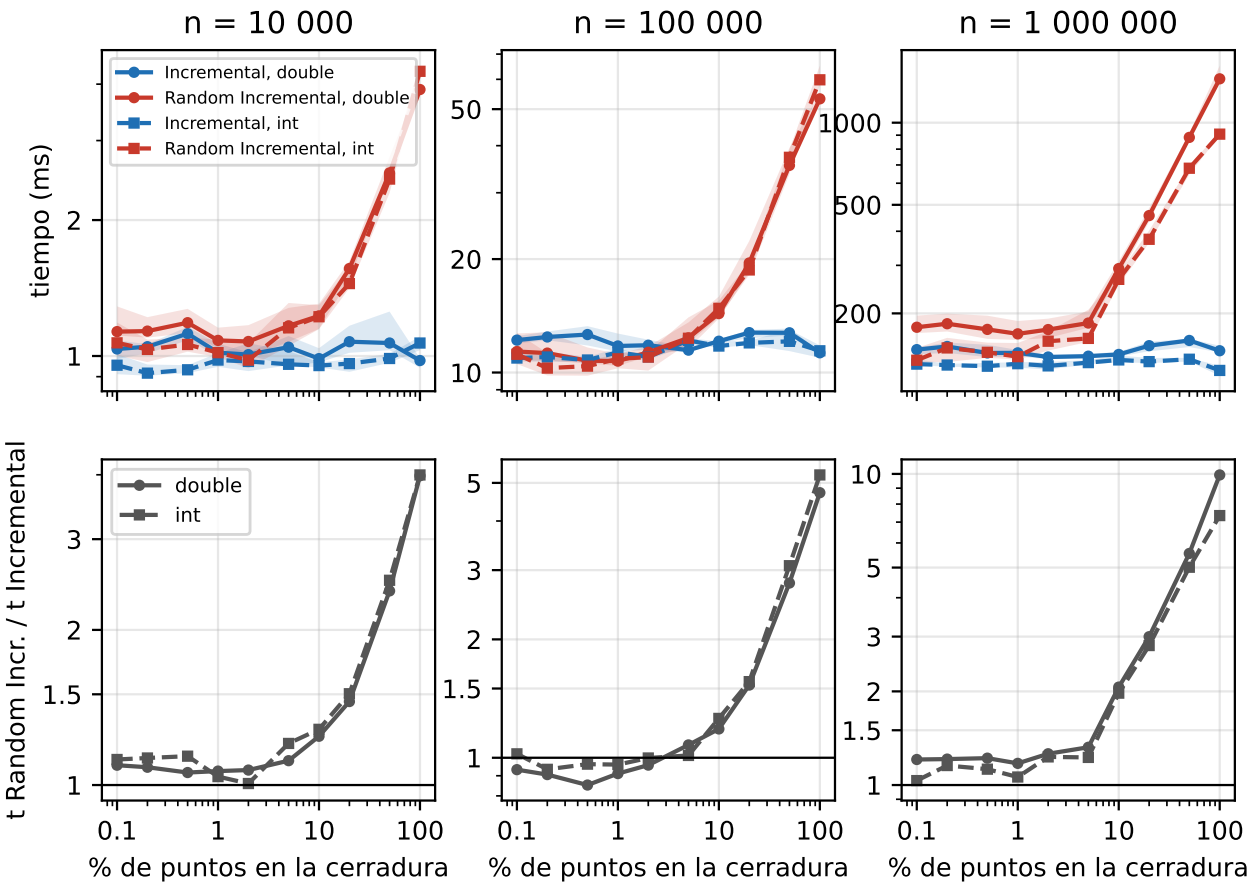


Figura 2: Arriba: tiempo contra el porcentaje de puntos en la cerradura (circunferencia), mediana de 10 nubes. Abajo: cociente entre las medianas; bajo la línea en 1 el Random Incremental es más rápido.

La Figura 2 muestra lo que se esperaba de los mecanismos: el Incremental es plano en el porcentaje, y el Random Incremental es plano hasta cerca de un 5 % y luego crece, hasta ser 9.39 veces más lento con todos los puntos en la cerradura y $n = 10^6$ (8.22 con `int`).

Puntos aleatorios (P2). Con $n = 10^6$ puntos uniformes la cerradura tiene una mediana de 36 vértices (se esperaban unos 37), menos de un 0.004 %. Los dos algoritmos tardan casi lo mismo, y cuál gana depende de n (Figura 1, abajo a la izquierda): en tamaños intermedios el Random Incremental es algo más rápido (el cociente $t_{\text{ran}}/t_{\text{inc}}$ llega a 0.90 con $n = 50\,000$ en `double` y a 0.84 con $n = 100\,000$ en `int`), pero con $n = 10^6$ es más lento: 172 contra 152 ms en `double` (cociente 1.14) y 155 contra 134 ms en `int` (1.16).

El porcentaje de cruce (P3). La Tabla 2 interpola, para cada n, el porcentaje en que el cociente cruza 1. El Random Incremental es más rápido solo con $n = 10^5$ y menos de un 2–3 % de los puntos en la cerradura, y por poco (un 7 % con `double`). Con $n = 10^4$ y $n = 10^6$ el Incremental es más rápido en

todos los porcentajes. Es la misma ventana de tamaños intermedios que se ve con los puntos uniformes, medidos en siete tamaños.

Tabla 2: Cociente de tiempos $t_{\text{ran}}/t_{\text{inc}}$ en los extremos del rango de porcentajes, y porcentaje en que cruza 1.

Tipo	n	con 0.1 %	con 100 %	Cruce
double	10 000	1.09	3.98	no se cruzan
double	100 000	0.93	4.72	2.8 %
double	1 000 000	1.21	9.92	no se cruzan
int	10 000	1.12	4.00	no se cruzan
int	100 000	1.02	5.24	2.2 %
int	1 000 000	1.03	7.35	no se cruzan

¿Por qué el cruce depende así de n? Con un porcentaje pequeño casi todos los puntos quedan dentro después de pocas inserciones y se descartan: el Random Incremental hace entre 4.1 y 4.3 reasignaciones por punto, *sin importar* n, es decir, trabajo lineal. El Incremental sigue haciendo $n \log n$. Escribiendo $t_{\text{inc}} = c_{\text{inc}}\, n \log_2 n$ y $t_{\text{ran}} = c_{\text{ran}}\, R$, con R las reasignaciones, el cociente se separa en dos factores:

$$\frac{t_{\text{ran}}}{t_{\text{inc}}} = \underbrace{\frac{R/n}{\log_2 n}}_{\text{operaciones}} \cdot \underbrace{\frac{c_{\text{ran}}}{c_{\text{inc}}}}_{\text{costo por operación}}.$$

La Tabla 3 muestra los dos factores. El de operaciones baja de 0.32 a 0.20 al pasar de 10^4 a 10^6, porque R/n es constante y $\log_2 n$ crece: favorece cada vez más al Random Incremental. Pero una reasignación cuesta entre 3 y 4 veces lo que cuesta una unidad de $n \log_2 n$ del Incremental mientras los datos caben en el caché, y casi 6 veces con $n = 10^6$. Con $n = 10^5$ el primer factor ya bajó lo suficiente y el segundo todavía no sube: ahí está la única ventana en que gana el Random Incremental.

Tabla 3: Descomposición del cociente de tiempos con un 0.1 % de los puntos en la cerradura (medianas de 10 nubes). c_{inc} en ns por $n \log_2 n$, c_{ran} en ns por reasignación.

Tipo	n	$\log_2 n$	R/n	$\frac{R/n}{\log_2 n}$	c_{inc}	c_{ran}	$\frac{c_{\text{ran}}}{c_{\text{inc}}}$	$\frac{t_{\text{ran}}}{t_{\text{inc}}}$
double	10 000	13.3	4.31	0.32	7.8	26.3	3.4	1.09
double	100 000	16.6	4.20	0.25	7.3	27.0	3.7	0.93
double	1 000 000	19.9	4.08	0.20	7.4	43.6	5.9	1.21
int	10 000	13.3	4.31	0.32	7.2	24.8	3.5	1.12
int	100 000	16.6	4.20	0.25	6.6	26.6	4.0	1.02
int	1 000 000	19.9	4.08	0.20	6.5	33.0	5.1	1.03

Entonces el porcentaje de cruce sí se puede determinar, pero no es un número fijo: depende de n y de la máquina, y aquí existe solo en una ventana alrededor de $n = 10^5$. La descomposición sugiere una hipótesis para otro ciclo de experimentos (no se midió): con n mucho mayor, cuando todos los accesos del Random Incremental ya fallan en el caché, $c_{\text{ran}}/c_{\text{inc}}$ dejaría de crecer mientras $\log_2 n$ sigue creciendo, y el Random Incremental volvería a ganar con porcentajes pequeños.

4.4. Variabilidad

La Tabla 4 separa dos fuentes de variación. La del algoritmo se ve en las reasignaciones, que son exactas: con puntos uniformes varían un 20 % entre semillas (dependen de cuán grande sea la cerradura

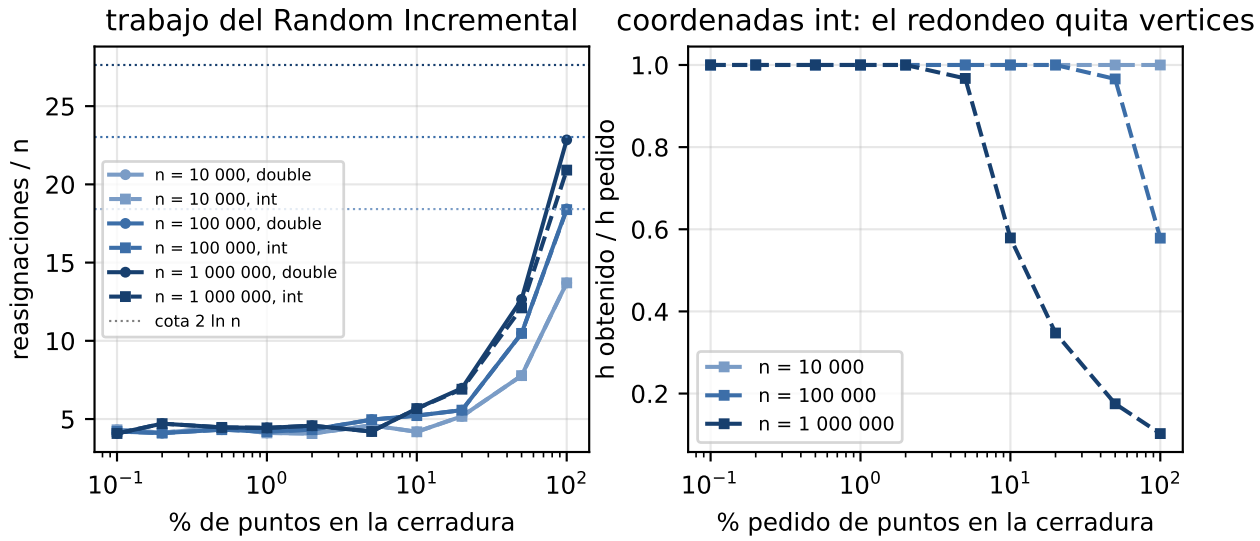


Figura 3: Izquierda: reasignaciones por punto del Random Incremental, contra el porcentaje en la cerradura; las líneas punteadas son la cota $2 \ln n$ de cada n. Derecha: con coordenadas `int`, fracción de los puntos pedidos en la circunferencia que siguen siendo vértices después de redondear.

Tabla 4: Una misma nube de 10^6 puntos medida con 20 semillas del barajado (el Incremental no usa la semilla: su variación es solo de la máquina). Tiempos en ms; CV es el coeficiente de variación.

Tipo	Nube	Algoritmo	Media	Desv.	CV	Reasignaciones	CV
double	uniforme	Incremental	155.1	12.5	8.1 %	–	–
double	uniforme	Random Incremental	199.8	34.8	17.4 %	4 227 082	19.99 %
double	100 % en la cerradura	Incremental	169.3	12.4	7.3 %	–	–
double	100 % en la cerradura	Random Incremental	1804.3	202.3	11.2 %	22 918 649	2.23 %
int	uniforme	Incremental	134.4	8.2	6.1 %	–	–
int	uniforme	Random Incremental	146.6	24.8	16.9 %	4 227 082	19.99 %
int	100 % en la cerradura	Incremental	123.0	8.5	6.9 %	–	–
int	100 % en la cerradura	Random Incremental	990.6	97.2	9.8 %	20 994 650	2.43 %

de los primeros puntos insertados, que es lo que permite descartar a los demás), y el tiempo sigue esa variación (correlación 0.70 con `double`, 0.74 con `int`). Con todos los puntos en la cerradura las reasignaciones varían solo un 2 %, y la variación del tiempo, de un 10 %, no se correlaciona con ellas (0.00 y -0.08): es de la máquina. La misma nube con la misma semilla, medida al principio y al final del experimento, difiere entre 3 y 29 %; el regulador de frecuencia de un notebook no se puede fijar sin permisos de administrador. Por eso los resultados se apoyan en medianas de comparaciones pareadas (los dos algoritmos sobre la misma nube, uno después del otro) y en las cuentas de operaciones, y una diferencia menor a un 10 % entre los algoritmos, como la ventana de $n = 10^5$, debe leerse con cautela.

4.5. Enteros y reales (P4)

Velocidad. Con la misma carga de trabajo (puntos uniformes, misma h), `int` es más rápido: el Incremental tarda 0.88 y el Random Incremental 0.90 veces lo que tardan con `double` con $n = 10^6$. Una razón es la memoria: `Punto<T>` guarda también la coordenada z de la Tarea 1, así que un `Punto<int>` ocupa 12 bytes y un `Punto<double>` 24, y ambos algoritmos mueven puntos completos.

Exactitud. Con `int` todas las orientaciones son exactas (por el rango de coordenadas de los supuestos), y las dos cerraduras coincidieron en todos los casos, también con puntos colineales. Con `double` solo coinciden mientras ninguna orientación dudosa decida algo (Sección 4.6).

El redondeo cambia la entrada. Al redondear puntos de la circunferencia, dos vecinos quedan separados de la cuerda de sus vecinos por apenas $R(1 - \cos\frac{2\pi}{k}) \approx 2\pi^2 R/k^2$, y el redondeo los mueve hasta medio entero. Con $R = 10^8$ esa separación baja de 1 cuando $k > \pi\sqrt{2R} \approx 44\,000$, y entonces muchos puntos dejan de estar en posición convexa. La Figura 3 (derecha) lo confirma: las pérdidas empiezan con $k = 50\,000$ (5 % de 10^6 y 50 % de 10^5), y con $n = 10^6$ y 100 % pedido solo quedan 102 550 vértices. No es un problema del generador sino de los enteros: en una grilla de $N \times N$ caben a lo más del orden de $N^{2/3}$ puntos en posición estrictamente convexa [5]. Con `int` el porcentaje real se mide en la salida y es el que corresponde usar.

4.6. Puntos colineales (P5)

La Tabla 5 tiene tres situaciones:

- **Colinealidad exacta** (cuadrado sin rotar, `int` y `double`): los puntos de los lados tienen exactamente la misma x o la misma y, las orientaciones dan 0 sin error, y ambos algoritmos entregan las 4 esquinas. Las reglas de la Sección 2.4 (sacar con orientación ≥ 0; prueba estricta y recorrido no estricto) bastan.
- **Sin colineales** (circunferencia): siempre coinciden, también con `double`, porque los vértices están lejos de ser colineales comparados con el error de redondeo.
- **Colinealidad que el tipo no representa** (cuadrado rotado en 30°): con `int` los puntos redondeados ya no son colineales, pero las orientaciones son exactas y ambos algoritmos calculan la misma cerradura, la de los puntos enteros que sobresalen de cada lado (25 a 58 vértices). Con `double` los puntos quedan a un redondeo de la recta, las orientaciones entre ellos son del tamaño del error, y las cerraduras *nunca* coinciden.

Para saber cuál de las dos es la correcta en el último caso, se recalculó la cerradura de los mismos `double` con aritmética racional exacta (`analisis/exactitud.py`, con `fractions.Fraction`; cada `double` es un racional). La Tabla 6 muestra que *ninguno* de los dos algoritmos entrega la cerradura exacta, y que fallan de maneras opuestas. El Incremental agrega vértices que no lo son: cuando el signo

Tabla 5: Cerraduras con un 10 % de los puntos en el borde, 10 nubes por fila: porcentaje de nubes con cerraduras idénticas y rango del número de vértices de cada algoritmo.

Nube	Tipo	n	Idénticas	h Incremental	h Random Incremental
cuadrado, 0°	int	10 000	100 %	4	4
cuadrado, 0°	int	100 000	100 %	4	4
cuadrado, 0°	int	1 000 000	100 %	4	4
cuadrado, 0°	double	10 000	100 %	4	4
cuadrado, 0°	double	100 000	100 %	4	4
cuadrado, 0°	double	1 000 000	100 %	4	4
cuadrado, 30°	int	10 000	100 %	25–35	25–35
cuadrado, 30°	int	100 000	100 %	36–47	36–47
cuadrado, 30°	int	1 000 000	100 %	41–58	41–58
cuadrado, 30°	double	10 000	0 %	21–29	10–16
cuadrado, 30°	double	100 000	0 %	39–55	14–21
cuadrado, 30°	double	1 000 000	0 %	46–67	19–26
circunferencia	int	10 000	100 %	1 000	1 000
circunferencia	int	100 000	100 %	10 000	10 000
circunferencia	int	1 000 000	100 %	57 752–58 092	57 752–58 092
circunferencia	double	10 000	100 %	1 000	1 000
circunferencia	double	100 000	100 %	10 000	10 000
circunferencia	double	1 000 000	100 %	100 000	100 000

de una orientación casi nula sale mal, cree ver una vuelta a la derecha y deja en la cadena un punto que debía sacar. El Random Incremental pierde vértices verdaderos: el mismo error en la prueba "está fuera" descarta un punto, y el descarte es para siempre. Además, como el resultado del Random Incremental depende del orden de inserción, con `double` cambia con la semilla.

Tabla 6: Cuadrado rotado en 30° con `double`, comparado con la cerradura exacta de los mismos puntos (10 nubes por fila): nubes en que la cerradura calculada es la exacta, y promedio de vértices exactos que faltan y de vértices calculados que sobran.

		Incremental			Random Incremental		
n	h exacta	Exactas	Faltan	Sobran	Exactas	Faltan	Sobran
10 000	17–23	0/10	7.0	11.0	0/10	9.1	1.3
100 000	26–38	0/10	12.5	27.0	0/10	15.1	1.9

Como pide el enunciado, los algoritmos no se hicieron consistentes en este caso: hacerlo requiere predicados exactos (por ejemplo, los adaptativos de Shewchuk) o coordenadas enteras. Con los supuestos de este trabajo, la respuesta práctica es usar coordenadas enteras cuando la entrada tiene colinealidades.

5. Conclusiones

P1 Ambos algoritmos hacen $\Theta(n \log n)$ operaciones (el Random Incremental en su peor caso, con todos los puntos en la cerradura). El Incremental también lo cumple en tiempo, con unos 7 ns por $n \log_2 n$; el Random Incremental se aleja de $n \log n$ en tiempo porque cada operación se encarece

cuando los datos no caben en el caché.

P2 Con puntos uniformes los dos tienen tiempos parecidos: el Random Incremental es hasta un 10–16 % más rápido en tamaños intermedios y un 14–16 % más lento con $n = 10^6$.

P3 El Incremental no depende del porcentaje de puntos en la cerradura; el Random Incremental hace trabajo lineal hasta un 5 % y luego crece hasta ser casi 10 veces más lento con el 100 %. El Random Incremental gana solo en tamaños intermedios (aquí, $n = 10^5$) y con menos de un 2–3 % en la cerradura; el porcentaje de cruce se puede determinar experimentalmente, pero depende de n y de la máquina, por la competencia entre $\log n$ y el costo de memoria (Tabla 3).

P4 Con la misma entrada, `int` es un 10–12 % más rápido y hace exactas todas las decisiones. Pero el redondeo cambia la entrada: al redondear una circunferencia de radio 10^8, desde unos 44 000 puntos estos dejan de estar en posición convexa.

P5 Con colinealidad exacta o sin colineales, ambas cerraduras coinciden siempre; con puntos casi colineales en `double` nunca coinciden, y ninguna es la exacta: el Incremental agrega vértices y el Random Incremental los pierde.

En la práctica, el Incremental es la mejor opción general: es más simple, su tiempo no depende de la entrada y es más rápido en casi todos los casos medidos.

Referencias

[1] G. Miller, J. Li. *Lecture Notes: Quicksort and Random Incremental Convex Hull*. 15-451 Algorithms, Carnegie Mellon University. https://www.cs.cmu.edu/~15451-f20/LectureNotes/2D-ConvexHull.pdf

[2] M. de Berg, O. Cheong, M. van Kreveld, M. Overmars. *Computational Geometry: Algorithms and Applications*, 3.ª ed. Springer, 2008. Sección 1.1.

[3] B. M. E. Moret. *Towards a Discipline of Experimental Algorithmics*. DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 59, 2002.

[4] A. Rényi, R. Sulanke. Über die konvexe Hülle von n zufällig gewählten Punkten. *Z. Wahrscheinlichkeitstheorie*, 2:75–84, 1963.

[5] V. Jarník. Über die Gitterpunkte auf konvexen Kurven. *Math. Z.*, 24:500–518, 1926.